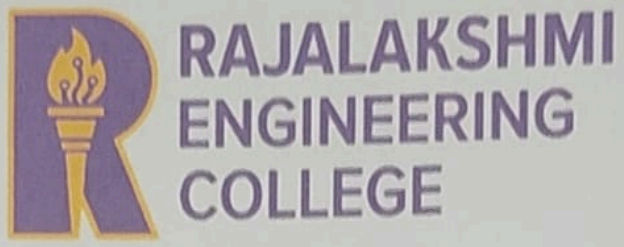
RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name :	VISHWADHARINI. A
Year / Branch / Section :	II - YEAR B.E. CSE - 'H'
University Register No. :	2116250701851
College Roll No. :	250701851
Semester :	III
Academic Year :	2026 - 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	VISHNADHARINI. A
ROLL NO	211625070185 1
YEAR	II - YEAR
SEC	CSE - 'H'

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ...VISHWADHARINI...A.....

Academic Year: 2026-2027.. **Semester:** ....III..... **Branch:** B.E.CSE.-H

Register No. 211625070185 1

Certified that this is the bonafide record of work done by the above student in the..CS2333.H...FUNDAMENTALS...OF..DATA...SCIENCE.... Laboratory during the academic year 2026 – 2027.

21/11/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

EXP NO: 1.A ANALYSE THE TREND OF DATA SCIENCE JOB POSTINGS OVER THE LAST DECADE

AIM:

Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

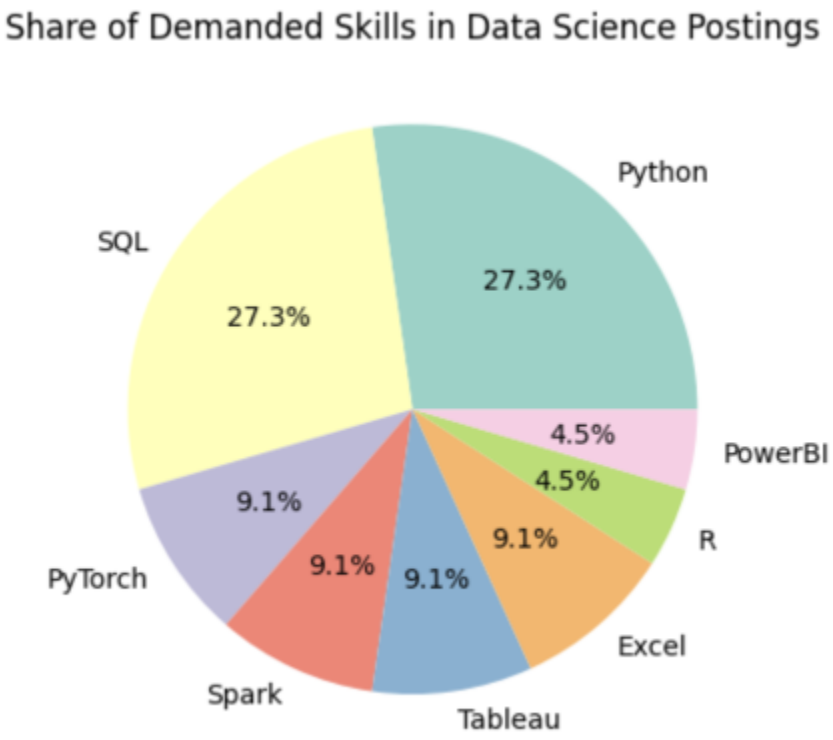
data = {
    'Job_Title': [
        'Data Scientist', 'Data Engineer', 'Data Analyst',
        'ML Engineer', 'Data Scientist', 'Data Analyst',
        'Data Engineer', 'Data Scientist'
    ],
    'Skills': [
        'Python, SQL, PyTorch',
        'Python, SQL, Spark',
        'SQL, Tableau, Excel',
        'Python, PyTorch',
        'Python, R, SQL',
        'Excel, SQL, PowerBI',
        'Python, Spark',
        'Python, SQL, Tableau'
    ]
}

df = pd.DataFrame(data)
skills_series = df['Skills'].str.split(', ').explode()
skill_counts = skills_series.value_counts()

plt.pie(
    skill_counts.values,
    labels=skill_counts.index,
    autopct='%1.1f%%',
    colors=plt.cm.Set3.colors
)
```

```
plt.title('Share of Demanded Skills in Data Science Postings')
plt.show()
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 1.B ANALYSE AND VISUALIZE THE DISTRIBUTION OF VARIOUS DATA SCIENCE ROLES FROM A DATA SET

AIM:

Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots

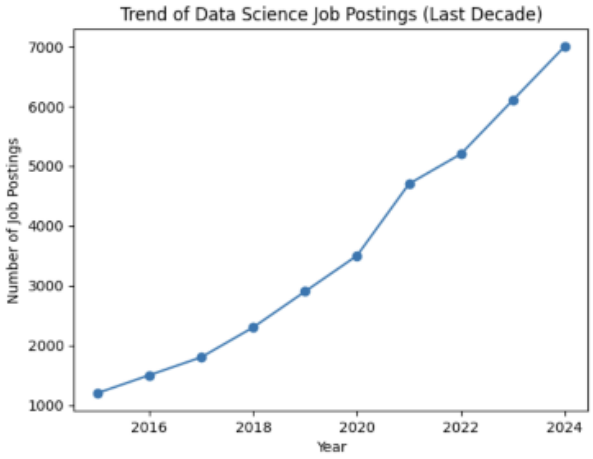
PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'Year': [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
    'Job_Postings': [1200, 1500, 1800, 2300, 2900, 3500, 4700, 5200, 6100, 7000]
}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job_Postings'], marker='o')
plt.title('Trend of Data Science Job Postings (Last Decade)')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 1.C CONDUCT AN EXPERIMENT TO DIFFERENTIATE STRUCTURED, UN-STRUCTURED AND SEMI STRUCTURED DATA BASED ON DATA SET GIVEN

AIM:

Create small datasets for each type and explain their characteristics.

PYTHON CODE:

```
import pandas as pd

structured = pd.DataFrame({
   "Name": ["Alice", "Bob"],
   "Age": [20, 21]
})
print(structured)


semi = {
   "Name": "Alice",
   "Age": 20
}
print(semi)

text = "Artificial Intelligence is changing the world."
print(text)
```

OUTPUT:

```
Name  Age
0  Alice   20
1    Bob   21
{'Name': 'Alice', 'Age': 20}
Artificial Intelligence is changing the world.
```

RESULT:

Thus the code is executed successfully.

EXP NO: 1.D CONDUCT AN EXPERIMENT TO ENCRYPT AND DECRYPT GIVEN SENSITIVE DATA

AIM:

Use the cryptography library to encrypt and decrypt a piece of data.

PYTHON CODE:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
f = Fernet(key)

data = "PIN-8#290"
print("Original Data:", data)

encrypted = f.encrypt(data.encode())
print("\nEncrypted Data:", encrypted)

decrypted = f.decrypt(encrypted).decode()
print("\nDecrypted Data:", decrypted)
```

OUTPUT:

```
Original Data: My ATM PIN is 1234

Encrypted Data: b'gAAAAABqchkp4FK4b5PjhhF2ZIW6jbLJ7zET9w3-
U57owpV_ICEw63YvtkpO5yTotmcgb5imtJCSlwyDQLmXXi8ABkVvkx5xJfmL
dVcfgBReBxPNikJbxlA='
```

RESULT:

Thus the code is executed successfully.

EXP NO: 2 UPLOAD AND ANALYZE THE DATA SET GIVEN IN CSV FORMAT AND PERFORM DATA PREPROCESSING AND VISUALIZATION

AIM:

Use sample data set sales-data.csv.

DATASET GIVEN:

Date	Product	Sales	Quantity	Region
01-01-2023	Product A	200	4	North
02-01-2023	Product B	150	3	South
03-01-2023	Product A	220	5	North
04-01-2023	Product C	300	6	East
05-01-2023	Product B	180	4	West
06-01-2023	Product A	210	5	North
07-01-2023	Product C	320	7	East
08-01-2023	Product B	160	3	South
09-01-2023	Product A	230	6	North
10-01-2023	Product C	310	7	East
11-01-2023	Product B	190	4	West
12-01-2023	Product A	240	6	North
13-01-2023	Product C	330	8	East
14-01-2023	Product B	170	3	South
15-01-2023	Product A	250	7	North
16-01-2023	Product C	340	8	East

PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_excel("sales_data.xlsx")
print(df.head())
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True, errors="coerce")
```

```
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["Sales"] = df["Sales"].fillna(df["Sales"].median())
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Product"] = df["Product"].fillna(df["Product"].mode()[0])
df["Region"] = df["Region"].fillna(df["Region"].mode()[0])
df = df.drop_duplicates()
df = df.dropna(subset=["Date"])
df = df.sort_values("Date")
print("\nCleaned Dataset:")
print(df)
product_sales = df.groupby("Product")["Sales"].sum()
plt.figure(figsize=(7, 7))
plt.pie(
    product_sales,
    labels=product_sales.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Product vs Sales")
plt.show()
region_sales = df.groupby("Region")["Sales"].sum().sort_values()
plt.figure(figsize=(8, 5))
colors = ["#4E79A7", "#F28E2B", "#59A14F", "#B07AA1"]
plt.barh(region_sales.index, region_sales.values, color=colors)
plt.title("Sales by Region")
plt.xlabel("Total Sales")
plt.ylabel("Region")
plt.tight_layout()
plt.show()
monthly_sales = df.groupby(df["Date"].dt.to_period("M"))["Sales"].sum()
plt.figure(figsize=(10, 5))
monthly_sales.plot(kind="line", marker="o")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT:

```
        Date    Product  Sales  Quantity Region
0 2023-01-01  Product A    200         4  North
1 2023-01-02  Product B    150         3  South
2 2023-01-03  Product A    220         5  North
3 2023-01-04  Product C    300         6   East
4 2023-01-05  Product B    180         4   West

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Date      16 non-null     datetime64[ns]
 1   Product   16 non-null     object
 2   Sales     16 non-null     int64
 3   Quantity  16 non-null     int64
 4   Region    16 non-null     object
dtypes: datetime64[ns](1), int64(2), object(2)

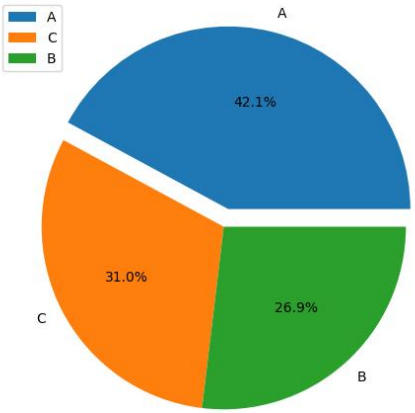
None

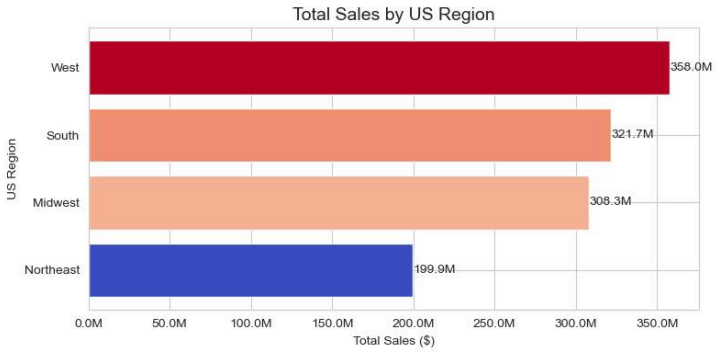
Missing Values:
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64

Duplicate Rows:
0

Cleaned Dataset:
        Date    Product  Sales  Quantity Region
0  2023-01-01  Product A    200         4  North
1  2023-01-02  Product B    150         3  South
2  2023-01-03  Product A    220         5  North
3  2023-01-04  Product C    300         6   East
4  2023-01-05  Product B    180         4   West
5  2023-01-06  Product A    210         5  North
6  2023-01-07  Product C    320         7   East
7  2023-01-08  Product B    160         3  South
8  2023-01-09  Product A    230         6  North
```

9	2023-01-10	Product C	310	7	East
10	2023-01-11	Product B	190	4	West
11	2023-01-12	Product A	240	6	North
12	2023-01-13	Product C	330	8	East
13	2023-01-14	Product B	170	3	South
14	2023-01-15	Product A	250	7	North
15	2023-01-16	Product C	340	8	East







RESULT:

Thus the code is executed successfully.

EXP NO: 3 HANDLING MISSING AND INAPPROPRIATE DATA IN A DATASET

AIM:

Demonstrate an experiment to handle missing data and inappropriate data in a Data set using Python Pandas Library for Data Preprocessing.

DATASET:

CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill
1	20-25	4	Ibis	veg	1300
2	30-35	5	LemonTree	Non-Veg	2000
3	25-30	6	RedFox	Veg	1322
4	20-25	-1	LemonTree	Veg	1234
5	35+	3	Ibis	Vegetarian	989
6	35+	3	Ibys	Non-Veg	1909
7	35+	4	RedFox	Vegetarian	1000
8	20-25	7	LemonTree	Veg	2999
9	25-30	2	Ibis	Non-Veg	3456
9	25-30	2	Ibis	Non-Veg	3456
10	30-35	5	RedFox	non-Veg	-6755

CODE:

```
import pandas as pd
import numpy as np
df = pd.read_excel("Hotel_Dataset.xlsx")
print("Original Dataset:")
print(df.head())
print("\nDataset Information:")
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:")
print(df.duplicated().sum())
df.drop_duplicates(inplace=True)
numeric_columns = df.select_dtypes(include=np.number).columns
for column in numeric_columns:
 df[column] = df[column].fillna(df[column].median())
categorical_columns = df.select_dtypes(include="object").columns
for column in categorical_columns:
```

```
 if df[column].isnull().any():
    df[column] = df[column].fillna(df[column].mode()[0])
for column in numeric_columns:
 Q1 = df[column].quantile(0.25)
 Q3 = df[column].quantile(0.75)
 IQR = Q3 - Q1
 lower_limit = Q1 - 1.5 * IQR
 upper_limit = Q3 + 1.5 * IQR
 df[column] = df[column].clip(lower_limit, upper_limit)
print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())
print("\nDuplicate Records After Preprocessing:")
print(df.duplicated().sum())
print("\nPreprocessed Dataset:")
print(df.head())
df.to_excel("Preprocessed_Hotel_Dataset.xlsx", index=False)
print("\nData preprocessing completed successfully.")
```

OUTPUT:

```
Original Dataset:
 CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax EstimatedSalary
Age_Group.1
0 1 20-25 4 Ibis veg 1300 2 40000 20-25
1 2 30-35 5 LemonTree Non-Veg 2000 3 59000 30-35
2 3 25-30 6 RedFox Veg 1322 2 30000 25-30
3 4 20-25 -1 LemonTree Veg 1234 2 120000 20-25
4 5 35+ 3 Ibis Vegetarian 989 2 45000 35+
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 # Column Non-Null Count Dtype
--- ------ -------------- -----
 0 CustomerID 11 non-null int64
 1 Age_Group 11 non-null str
 2 Rating(1-5) 11 non-null int64
 3 Hotel 11 non-null str
 4 FoodPreference 11 non-null str
 5 Bill 11 non-null int64
 6 NoOfPax 11 non-null int64
 7 EstimatedSalary 11 non-null int64
 8 Age_Group.1 11 non-null str
dtypes: int64(5), str(4)
```

memory usage: 924.0 bytes
Missing Values:
CustomerID 0
Age_Group 0
Rating(1-5) 0
Hotel 0
FoodPreference 0
Bill 0
NoOfPax 0
EstimatedSalary 0
Age_Group.1 0
dtype: int64
Duplicate Records:
1
/Users/yuva/Desktop/Code_Hub/FDS_Exp/Exp_3/exp.py:25: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-selectdtypes for details on how to writecode that works with pandas 2 and 3.
 categorical_columns = df.select_dtypes(include="object").columns
Missing Values After Preprocessing:
CustomerID 0
Age_Group 0
Rating(1-5) 0
Hotel 0
FoodPreference 0
Bill 0
NoOfPax 0
EstimatedSalary 0
Age_Group.1 0
dtype: int64
Duplicate Records After Preprocessing:
0
Preprocessed Dataset:
 CustomerID Age_Group Rating(1-5) Hotel FoodPreference Bill NoOfPax EstimatedSalary Age_Group.1
0 1 20-25 4 Ibis veg 1300.0 2.0 40000.0 20-25
1 2 30-35 5 LemonTree Non-Veg 2000.0 3.0 59000.0 30-35
2 3 25-30 6 RedFox Veg 1322.0 2.0 30000.0 25-30
3 4 20-25 0 LemonTree Veg 1234.0 2.0 120000.0 20-25
4 5 35+ 3 Ibis Vegetarian 989.0 2.0 45000.0 35+
Data preprocessing completed successfully

RESULT:

Thus the code is executed successfully.

EXP NO: 4 EXPERIMENT TO UNDERSTAND THE DATA PREPROCESSING IN DATA SCIENCE

AIM:

Understand the importance of Data preprocessing in data science

DATASET:

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40		Yes
France	35	58000	Yes
Spain		52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes

CODE:

```
import numpy as np
import pandas as pd

# Create DataFrame directly
data = {
    "Country": [
        "France",
        "Spain",
        "Germany",
        "Spain",
        "Germany",
        "France",
        "Spain",
        "France",
        "Germany",
        "France",
    ],
    "Age": [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    "Salary": [
```

```
        72000,
        48000,
        54000,
        61000,
        np.nan,
        58000,
        52000,
        79000,
        83000,
        67000,
    ],
    "Purchased": [
        "No",
        "Yes",
        "No",
        "No",
        "Yes",
        "Yes",
        "No",
        "Yes",
        "No",
        "Yes",
    ],
}
df = pd.DataFrame(data)
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
df = pd.get_dummies(df, columns=["Country"], dtype=int)
df["Purchased"] = np.where(df["Purchased"] == "Yes", 1, 0)
df["Age"] = (df["Age"] - np.mean(df["Age"])) / np.std(df["Age"])
df["Salary"] = (df["Salary"] - np.mean(df["Salary"])) / np.std(df["Salary"])

print(df)
```

OUTPUT:

```
        Age        Salary  Purchased  Country_France  Country_Germany  \
0  0.758874  7.494733e-01          0               1                0
1 -1.711504 -1.438178e+00          1               0                0
2 -1.275555 -8.912655e-01          0               0                1
3 -0.113024 -2.532004e-01          0               0                0
4  0.177609  6.632192e-16          1               0                1
5 -0.548973 -5.266569e-01          1               1                0
6  0.000000 -1.073570e+00          0               0                0
7  1.340140  1.387538e+00          1               1                0
8  1.630773  1.752147e+00          0               0                1
9 -0.258340  2.937125e-01          1               1                0

   Country_Spain
0              0
1              1
2              0
3              1
4              0
5              0
6              1
7              0
8              0
9              0
```

RESULT:

Thus the code is executed successfully.

EXP NO: 5 EXPERIMENT TO DETECT OUTLIERS IN A GIVEN DATA SET

AIM:

Understand the procedure to identify the outliers in a given dataset.

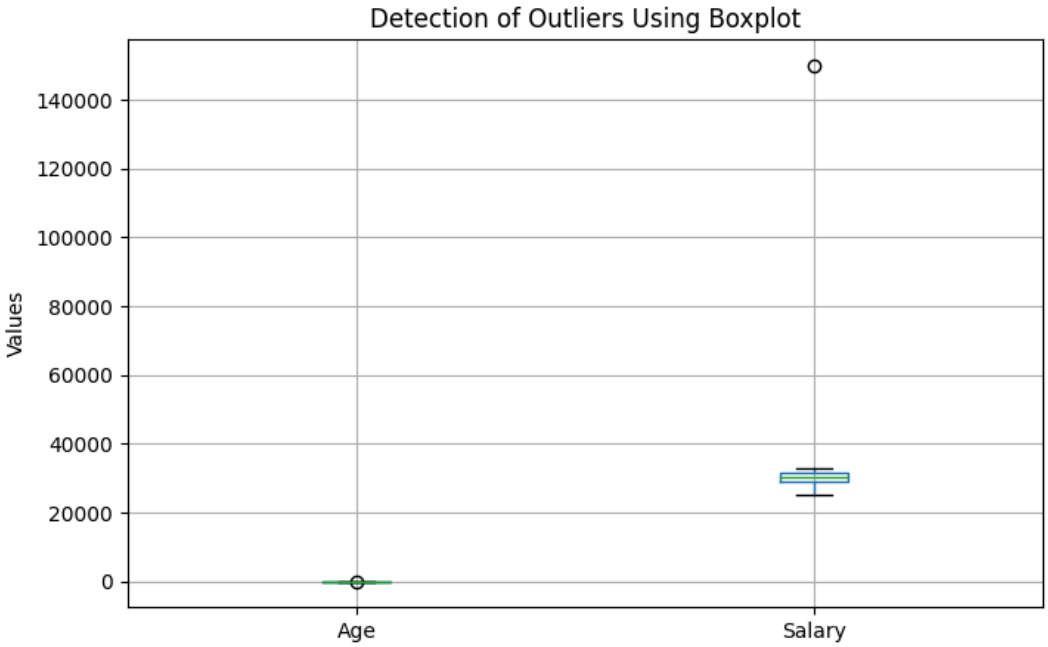
CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
 "Age": [20, 21, 22, 23, 21, 24, 22, 25, 23, 22, 21, 100],
 "Salary": [25000, 28000, 30000, 32000, 29000, 31000,
 30500, 33000, 31500, 29500, 28500, 150000]
}
df = pd.DataFrame(data)
for column in df.select_dtypes(include="number").columns:
 Q1 = df[column].quantile(0.25)
 Q3 = df[column].quantile(0.75)
 IQR = Q3 - Q1
 lower_bound = Q1 - 1.5 * IQR
 upper_bound = Q3 + 1.5 * IQR
 outliers = df[
 (df[column] < lower_bound) |
 (df[column] > upper_bound)
 ]
 print("\nColumn:", column)
 print("Q1:", Q1)
 print("Q3:", Q3)
 print("IQR:", IQR)
 print("Lower Bound:", lower_bound)
 print("Upper Bound:", upper_bound)
 print("Outliers:")
 print(outliers[[column]])
plt.figure(figsize=(8, 5))
df.boxplot(column=["Age", "Salary"])
plt.title("Detection of Outliers Using Boxplot")
plt.ylabel("Values")
plt.show()
```

OUTPUT:

```
Column: Age
Q1: 21.0
Q3: 23.25
IQR: 2.25
Lower Bound: 17.625
Upper Bound: 26.625
Outliers:
    Age
11  100

Column: Salary
Q1: 28875.0
Q3: 31625.0
IQR: 2750.0
Lower Bound: 24750.0
Upper Bound: 35750.0
Outliers:
    Salary
11  150000
```



RESULT:

Thus the code is executed successfully.

EXP NO: 6 EXPERIMENT TO UNDERSTAND FEATURE SCALING

AIM:

Understand the importance of feature scaling.

CODE:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler
data = {
 'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany',
 'France', 'Spain', 'France', None, 'France'],
 'Age': [44.0, 27.0, 30.0, 38.0, 40.0,
 35.0, None, 48.0, 50.0, 37.0],
 'Salary': [72000.0, 48000.0, 54000.0, 61000.0, None,
 58000.0, 52000.0, 79000.0, 83000.0, 67000.0],
 'Purchased': ['No', 'Yes', 'No', 'No', 'Yes',
 'Yes', 'No', 'Yes', 'No', 'Yes']
}
df = pd.DataFrame(data)
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
X = df[['Age', 'Salary']]
standard_scaler = StandardScaler()
X_standardized = standard_scaler.fit_transform(X)
standardized_df = pd.DataFrame(
 X_standardized,
 columns=['Age', 'Salary']
)
print("\nStandardized Data:")
print(standardized_df)
minmax_scaler = MinMaxScaler()
X_normalized = minmax_scaler.fit_transform(X)
normalized_df = pd.DataFrame(
 X_normalized,
 columns=['Age', 'Salary']
)
print("\nNormalized Data:")
print(normalized_df)
```

OUTPUT:

```
Standardized Data:
        Age        Salary
0  0.758874  7.494733e-01
1 -1.711504 -1.438178e+00
2 -1.275555 -8.912655e-01
3 -0.113024 -2.532004e-01
4  0.177609  6.632192e-16
5 -0.548973 -5.266569e-01
6  0.000000 -1.073570e+00
7  1.340140  1.387538e+00
8  1.630773  1.752147e+00
9 -0.258340  2.937125e-01

Normalized Data:
        Age    Salary
0  0.739130  0.685714
1  0.000000  0.000000
2  0.130435  0.171429
3  0.478261  0.371429
4  0.565217  0.450794
5  0.347826  0.285714
6  0.512077  0.114286
7  0.913043  0.885714
8  1.000000  1.000000
9  0.434783  0.542857
```

RESULT:

Thus the code is executed successfully.

EXP NO: 7 EXPERIMENT TO UNDERSTAND EDA-QUANTITATIVE AND QUALITATIVE ANALYSIS

AIM:

Understand the importance of EDA-Quantitative and Qualitative analysis.

PYTHON CODE:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline

np.random.seed(42)

n = 250

shopping = pd.DataFrame({
   "Purchase_Amount": np.random.randint(200, 5000, n),
   "Discount": np.random.randint(0, 50, n),
   "Age": np.random.randint(18, 60, n),
   "Gender": np.random.choice(["Male", "Female"], n),
   "Category": np.random.choice(
      ["Electronics", "Clothing", "Home", "Sports"], n),
   "Payment": np.random.choice(
      ["UPI", "Card", "Cash"], n),
   "Delivery": np.random.choice(
      ["Standard", "Express"], n)
})

shopping["Final_Price"] = shopping["Purchase_Amount"] * (1 - shopping["Discount"]/100)

print(shopping.head())
print(shopping.info())
print(shopping.describe())
sns.jointplot(
   x="Discount",
   y="Final_Price",
   data=shopping
)
```

```
plt.show()
sns.jointplot(
    x="Age",
    y="Purchase_Amount",
    data=shopping,
    kind="hex"
)
plt.show()
plt.figure(figsize=(6,4))

sns.heatmap(
    shopping.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.show()
```

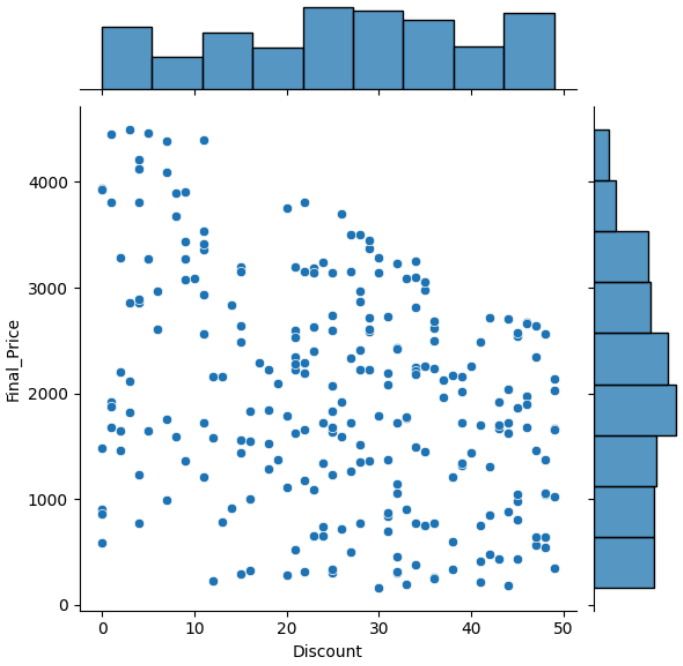
OUTPUT:

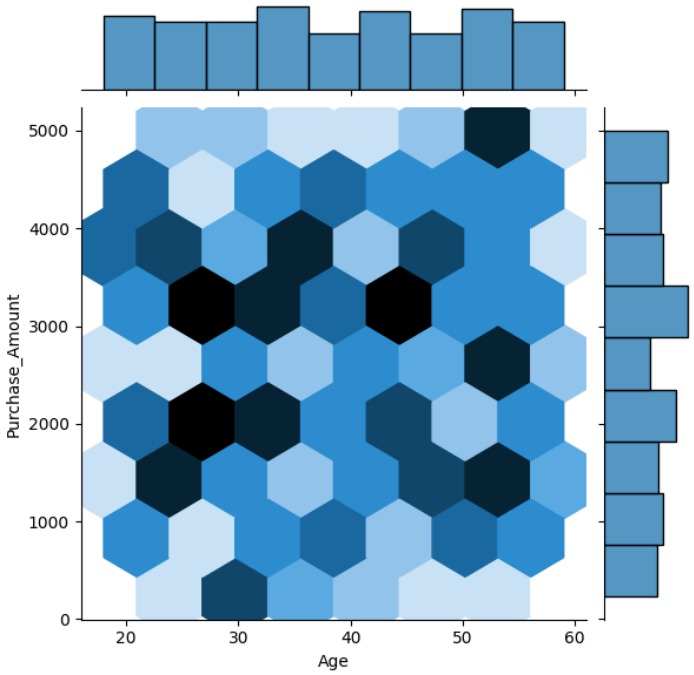
	Purchase_Amount	Discount	Age	Gender	Category	Payment	Delivery	\
0	1060	47	53	Male	Clothing	Cash	Express	
1	3972	11	35	Female	Sports	Card	Express	
2	3292	21	56	Male	Electronics	UPI	Express	
3	666	21	49	Female	Home	Cash	Standard	
4	4626	45	41	Male	Electronics	Card	Standard	

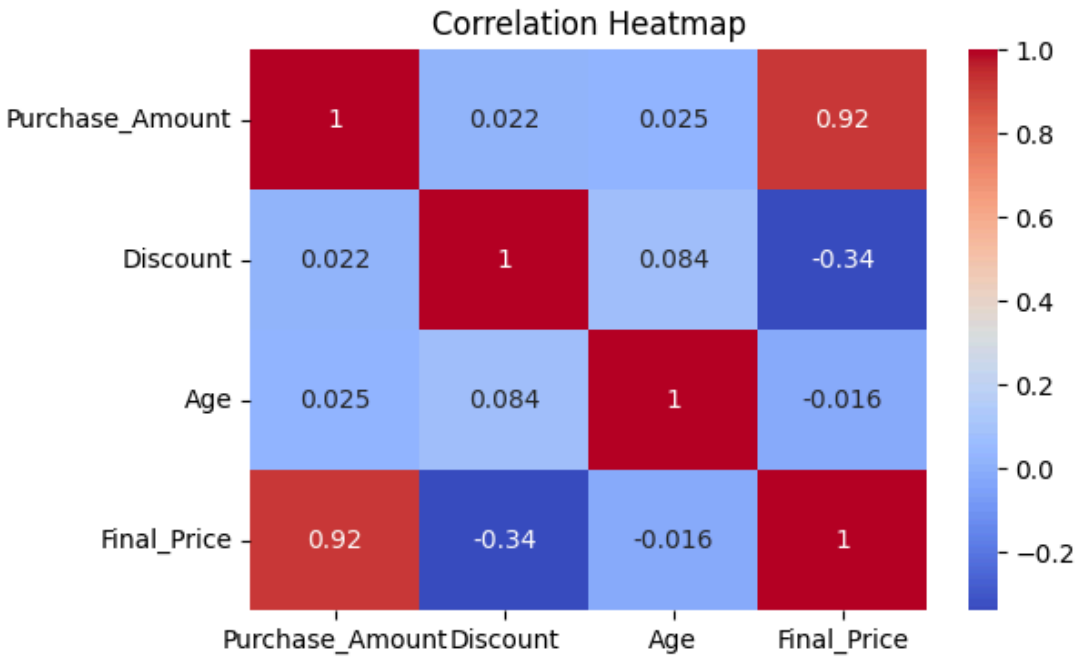
	Final_Price
0	561.80
1	3535.08
2	2600.68
3	526.14
4	2544.30

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250 entries, 0 to 249
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Purchase_Amount  250 non-null    int64
 1   Discount         250 non-null    int64
 2   Age              250 non-null    int64
 3   Gender           250 non-null    object
 4   Category         250 non-null    object
 5   Payment          250 non-null    object
 6   Delivery         250 non-null    object
 7   Final_Price      250 non-null    float64
dtypes: float64(1), int64(3), object(4)
memory usage: 15.8+ KB
None
```

	Purchase_Amount	Discount	Age	Final_Price
count	250.00000	250.000000	250.000000	250.000000
mean	2669.57200	25.964000	38.524000	1972.322680
std	1348.01858	14.057352	11.860358	1061.107602
min	234.00000	0.000000	18.000000	163.800000
25%	1564.50000	15.000000	29.000000	1152.510000
50%	2763.00000	27.000000	38.000000	1887.300000
75%	3759.75000	36.000000	49.000000	2701.040000
max	4998.00000	49.000000	59.000000	4494.010000







RESULT:

Thus the code is executed successfully.

EXP NO: 8 EXPERIMENT TO UNDERSTAND LINEAR REGRESSION FOR A GIVEN DATA SET

AIM:

Understand the Linear regression for the dataset given.

DATASET GIVEN:

YearsExperience	Salary
1.1	39343
1.3	46205
1.5	37731
2	43525
2.2	39891
2.9	56642
3	60150
3.2	54445
3.2	64445
3.7	57189
3.9	63218
4	55794
4	56957
4.1	57081
4.5	61111
4.9	67938
5.1	66029
5.3	83088
5.9	81363
6	93940
6.8	91738
7.1	98273
7.9	101302
8.2	113812
8.7	109431
9	105582
9.5	116969
9.6	112635
10.3	122391
10.5	121872

PYTHON CODE:

```
import numpy as np
import pandas as pd
data = {
   'YearsExperience': [
      1.1, 1.3, 1.5, 2, 2.2, 2.9, 3, 3.2, 3.2, 3.7,
      3.9, 4, 4, 4.1, 4.5, 4.9, 5.1, 5.3, 5.9, 6,
      6.8, 7.1, 7.9, 8.2, 8.7, 9, 9.5, 9.6, 10.3, 10.5
   ],
   'Salary': [
      39343, 46205, 37731, 43525, 39891, 56642, 60150, 54445,
      64445, 57189, 63218, 55794, 56957, 57081, 61111, 67938,
      66029, 83088, 81363, 93940, 91738, 98273, 101302,
      113812, 109431, 105582, 116969, 112635, 122391, 121872
   ]
}
df = pd.DataFrame(data)
df.info()
df.dropna(inplace=True)
df.info()
df.describe()
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
   features, label, test_size=0.2, random_state=42
)
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
model.score(x_train, y_train)
model.score(x_test, y_test)
model.coef_
model.intercept_

import pickle
pickle.dump(model, open('SalaryPred.model', 'wb'))
model = pickle.load(open('SalaryPred.model', 'rb'))
yr_of_exp = float(input("Enter Years of Experience: "))
yr_of_exp_NP = np.array([[yr_of_exp]])
Salary = model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}: ".format( yr_of_exp, Salary))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
Enter Years of Experience: 7
Estimated Salary for 7.0 years of experience is [[91288.29027299]]:
```

RESULT:

Thus the code is executed successfully.

EXP NO: 9 EXPERIMENT TO UNDERSTAND LOGISTIC REGRESSION FOR A GIVEN DATA SET

AIM:

Understand the Logistic Regression algorithm for the dataset given.

PYTHON CODE:

```
import numpy as np
import pandas as pd

df = pd.read_csv('Social_Network_Ads.csv')
df.head()
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values
features
label
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )

    model = LogisticRegression()
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(
            test_score, train_score, i
        ))
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=4
)
finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)
print(finalModel.score(x_train, y_train))
print(finalModel.score(x_test, y_test))
from sklearn.metrics import classification_report
print(classification_report(label, finalModel.predict(features)))
```

OUTPUT:

```
Test 0.8625 Train 0.8375 Random State 294
Test 0.8875 Train 0.828125 Random State 297
Test 0.8625 Train 0.834375 Random State 300
Test 0.8625 Train 0.85 Random State 301
Test 0.8875 Train 0.85 Random State 302
Test 0.875 Train 0.846875 Random State 303
Test 0.8625 Train 0.834375 Random State 305
Test 0.9125 Train 0.8375 Random State 306
Test 0.875 Train 0.846875 Random State 308
Test 0.9 Train 0.84375 Random State 311
Test 0.8625 Train 0.834375 Random State 313
Test 0.9125 Train 0.834375 Random State 314
Test 0.875 Train 0.8375 Random State 315
Test 0.9 Train 0.846875 Random State 317
Test 0.9125 Train 0.821875 Random State 319
Test 0.8625 Train 0.85 Random State 321
Test 0.9125 Train 0.828125 Random State 322
Test 0.85 Train 0.846875 Random State 328
Test 0.85 Train 0.8375 Random State 332
Test 0.8875 Train 0.853125 Random State 336
Test 0.85 Train 0.8375 Random State 337
Test 0.875 Train 0.840625 Random State 343
Test 0.8625 Train 0.84375 Random State 346
Test 0.8875 Train 0.83125 Random State 351
Test 0.8625 Train 0.85 Random State 352
Test 0.95 Train 0.81875 Random State 354
Test 0.8625 Train 0.85 Random State 356
Test 0.9125 Train 0.840625 Random State 357
Test 0.8625 Train 0.8375 Random State 358
Test 0.85 Train 0.840625 Random State 362
Test 0.9 Train 0.84375 Random State 363
Test 0.8625 Train 0.853125 Random State 364
Test 0.9375 Train 0.821875 Random State 366
Test 0.9125 Train 0.840625 Random State 369
Test 0.8625 Train 0.853125 Random State 371
Test 0.925 Train 0.834375 Random State 376
Test 0.9125 Train 0.828125 Random State 377
Test 0.8875 Train 0.85 Random State 378
Test 0.8875 Train 0.85 Random State 379
Test 0.8625 Train 0.840625 Random State 382
Test 0.8625 Train 0.859375 Random State 386
Test 0.85 Train 0.8375 Random State 387
Test 0.875 Train 0.828125 Random State 388
Test 0.85 Train 0.84375 Random State 394
Test 0.8625 Train 0.8375 Random State 395
Test 0.9 Train 0.84375 Random State 397
Test 0.8625 Train 0.84375 Random State 400
0.840625
0.9
              precision    recall  f1-score   support

           0       0.87      0.91      0.89       257
           1       0.82      0.75      0.78       143

    accuracy                           0.85       400
   macro avg       0.84      0.83      0.84       400
weighted avg       0.85      0.85      0.85       400
```

RESULT:

Thus the code is executed successfully.

EXP NO: 10 EXPERIMENT TO UNDERSTAND K-MEANS CLUSTERING ALGORITHM FOR GIVEN DATA SET

AIM:

Understand the K-means clustering algorithm for the dataset given.

PYTHON CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

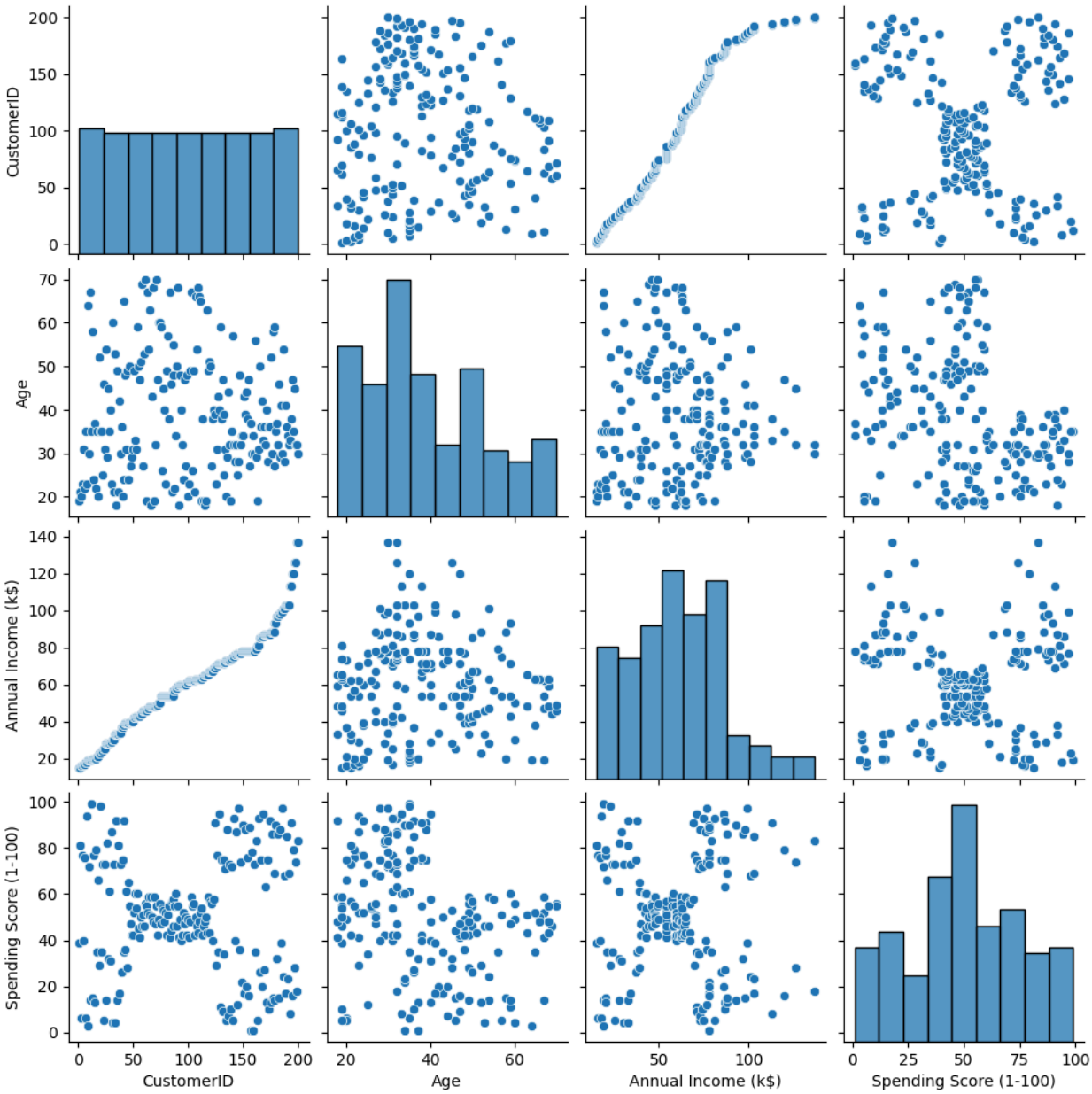
df = pd.read_csv('Mall_Customers.csv')
df.info()
df.head()
sns.pairplot(df)
features = df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model = KMeans(n_clusters=5)
model.fit(features)
Final = df.iloc[:,[3,4]]
Final['label'] = model.predict(features)
Final.head()
sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8) \
   .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
   .add_legend()

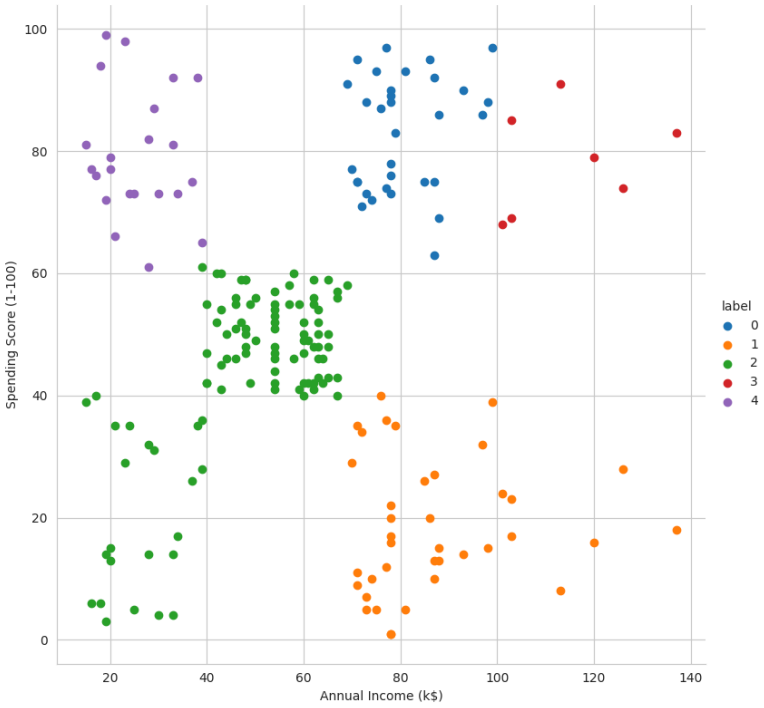
plt.show()
features_el = df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss = []
for i in range(1,10):
   model = KMeans(n_clusters=i)
   model.fit(features_el)
   wcss.append(model.inertia_)

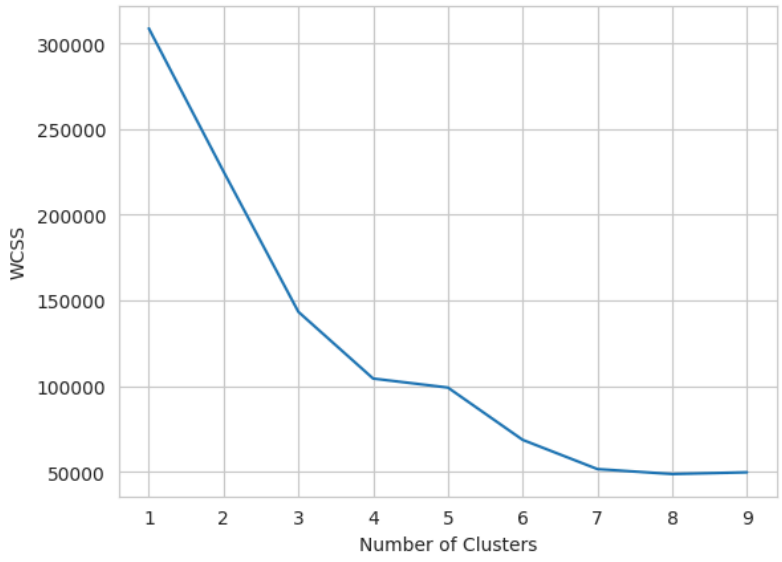
plt.plot(range(1,10),wcss)
plt.show()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```







RESULT:

Thus the code is executed successfully.

EXP NO: 11 EXPERIMENT TO UNDERSTAND KNN ALGORITHM FOR A GIVEN DATA SET

AIM:

Understand the KNN algorithm for the dataset given.

PYTHON CODE:

```
import numpy as np
import pandas as pd

df = pd.read_csv('Iris.csv')
df.info()
df.variety.value_counts()
df.head()
features = df.iloc[:,:-1].values
label = df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain, xtest, ytrain, ytest = train_test_split(
   features, label, test_size=.2, random_state=1
)
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
0.9583333333333334
1.0
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
   Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

RESULT:

Thus the code is executed successfully.

EXP NO: 12 CONDUCT Z TEST FOR THE DATA GIVEN USING PYTHON OBJECTIVE

AIM:

To test whether the average weight of a species of birds differs from 150 grams.

PYTHON CODE:

```
import numpy as np
import scipy.stats as stats

sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,151, 150,
149, 152, 151, 148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150,
153])
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly different from
150 grams.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in
average weight from 150 grams.")
```

OUTPUT:

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

RESULT:

Thus the code is executed successfully.

EXP NO: 13.A CONDUCT AN EXPERIMENT TO SHOW DATA VISUALIZATION USING LINE PLOT

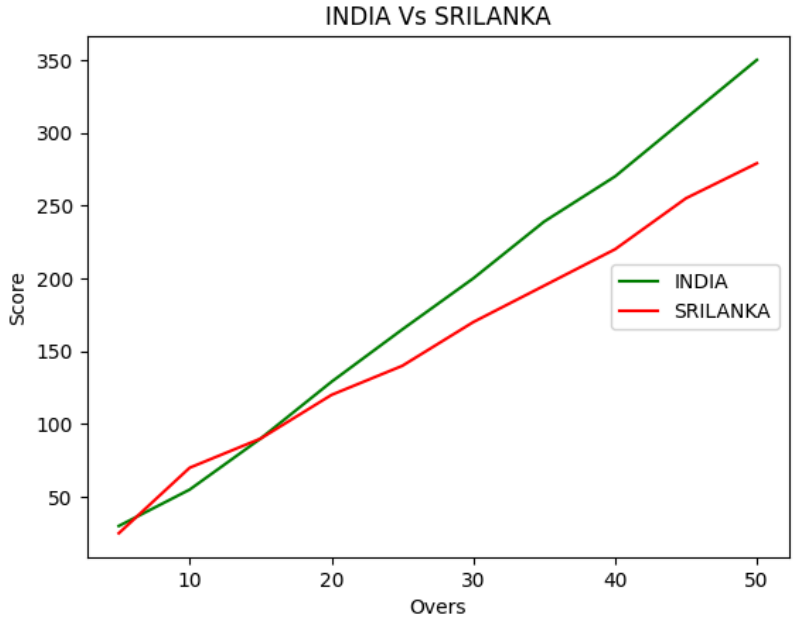
AIM:

Take any sample data either through csv file or data fetched directly through code.

PYTHON CODE:

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 13.B CONDUCT AN EXPERIMENT TO SHOW DATA VISUALIZATION USING BAR CHART

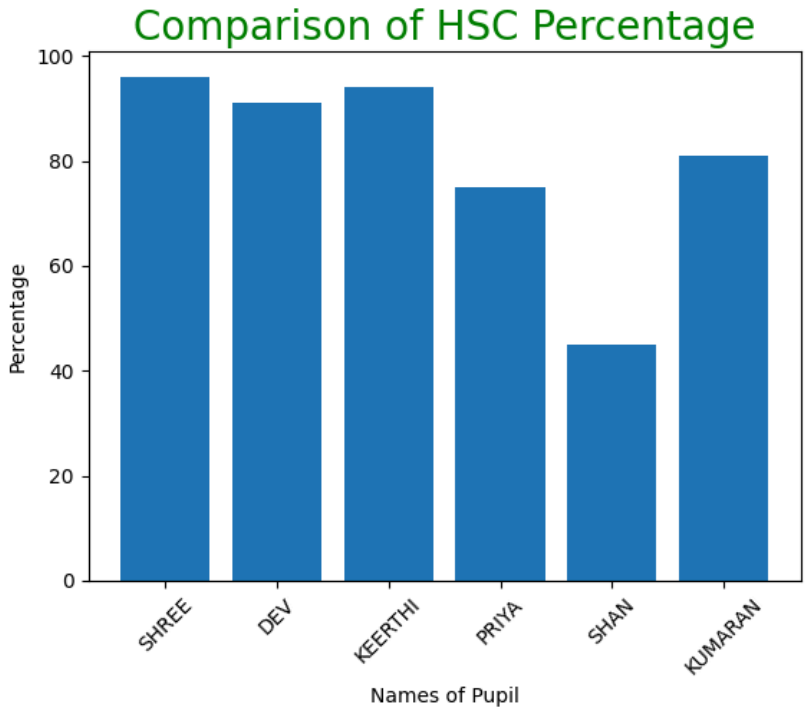
AIM:

Take any sample data either through csv file or data fetched directly through code.

PYTHON CODE:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 13.C CONDUCT AN EXPERIMENT TO SHOW DATA VISUALIZATION USING PIE CHART

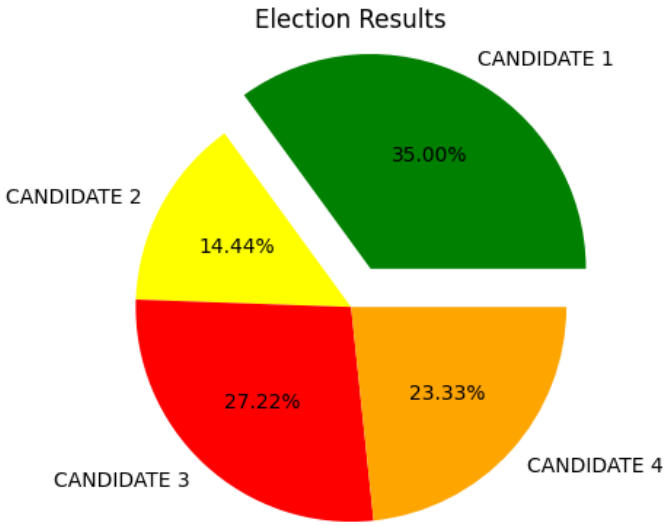
AIM:

Take any sample data either through csv file or data fetched directly through code.

PYTHON CODE:

```
import matplotlib.pyplot as election
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 14 TO COUNT THE FREQUENCY OF OCCURENCE OF A WORD IN A BODY OF TEXT IS OFTEN NEEDED DURING TEXT PROCESSING

AIM:

Conduct an experiment to show data visualization using line plot

PYTHON CODE:

```
text = """
Data science is an important field of study.
Data science combines statistics, mathematics,
programming, and machine learning.
Data scientists collect data from different sources.
They clean data, analyze data, and visualize data.
Python is widely used in data science.
Python provides libraries like NumPy, Pandas,
Matplotlib, and Scikit-learn.
These libraries help process large datasets.
Data analysis helps organizations make decisions.
Machine learning uses data to build predictive models.
Data visualization presents information through graphs.
Graphs include line plots, bar charts, and pie charts.
Data mining discovers useful patterns in data.
Big data contains large amounts of information.
Data engineers prepare data for analysis.
Data analysts study trends and summarize results.
Artificial intelligence uses data to learn patterns.
Good data improves the accuracy of predictions.
Data science is useful in healthcare, education,
banking, business, agriculture, and transportation.
Students learn data science to solve real-world problems.
The demand for data science professionals is increasing.
Data processing is an essential step in every project.
"""

word = input("Enter the word to count: ").lower()

words = text.lower().replace(".", "").replace(",", "").split()
frequency = words.count(word)
```

```
print("\nTotal words in the text:", len(words))
print("Word searched:", word)
print("Frequency of occurrence:", frequency)
```

OUTPUT:

```
Enter the word to count: data

Total words in the text: 131
Word searched: data
Frequency of occurrence: 15
```

RESULT:

Thus the code is executed successfully.

EXP NO: 15 DATA COLLECT AND INITIAL EXPLORATION

AIM:

To collect, load, and perform initial exploration of the diabetes dataset

PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_excel("Diabetes.xlsx")
print("First Five Rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nDataset Information:")
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
print("\nStatistical Summary:")
print(df.describe())
plt.hist(df["target"], bins=20, edgecolor="black")
plt.title("Distribution of Diabetes Target Values")
plt.xlabel("Target Value")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:

```
First Five Rows:
        age       sex       bmi  ...        s4        s5        s6
0  0.038076  0.050680  0.061696  ... -0.002592  0.019907 -0.017646
1 -0.001882 -0.044642 -0.051474  ... -0.039493 -0.068332 -0.092204
2  0.085299  0.050680  0.044451  ... -0.002592  0.002861 -0.025930
3 -0.089063 -0.044642 -0.011595  ...  0.034309  0.022688 -0.009362
4  0.005383 -0.044642 -0.036385  ... -0.002592 -0.031988 -0.046641
```

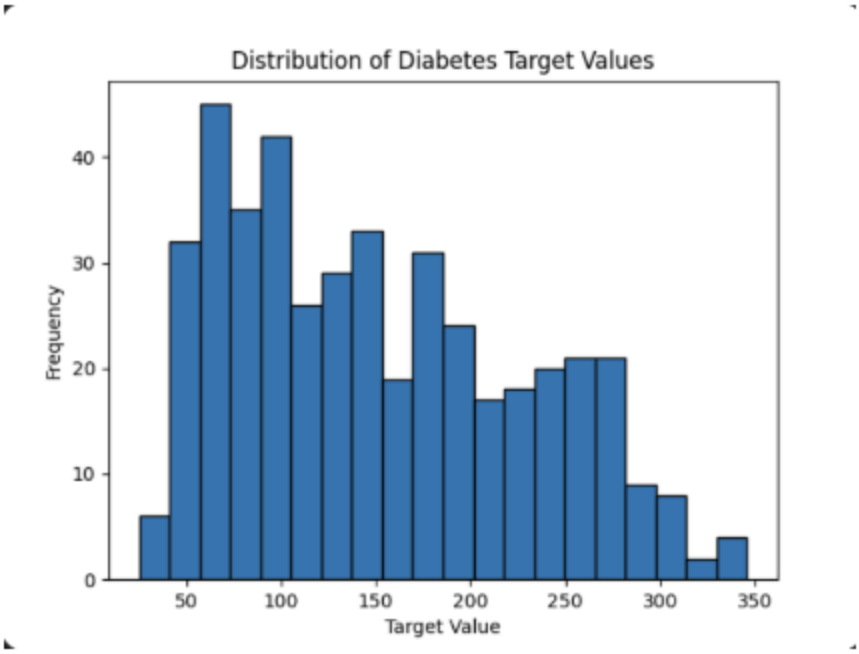
```
[5 rows x 10 columns]
Dataset Shape:(442, 10)

Column Names:Index(['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3',
's4', 's5', 's6'], dtype='object')

Dataset Information:<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 10 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
 2   bmi     442 non-null    float64
 3   bp      442 non-null    float64
 4   s1      442 non-null    float64
 5   s2      442 non-null    float64
 6   s3      442 non-null    float64
 7   s4      442 non-null    float64
 8   s5      442 non-null    float64
 9   s6      442 non-null    float64
dtypes: float64(10)
memory usage: 34.6 KB
None

Missing Values:age    0
sex    0
bmi    0
bp     0
s1     0
s2     0
s3     0
s4     0
s5     0
s6     0
dtype: int64
Statistical Summary:                age           sex  ...
s5            s6
count  4.420000e+02  4.420000e+02  ...  4.420000e+02  4.420000e+02
mean  -2.511817e-19  1.230790e-17  ...  9.293722e-17  1.130318e-17
std    4.761905e-02  4.761905e-02  ...  4.761905e-02  4.761905e-02
min   -1.072256e-01 -4.464164e-02  ... -1.260971e-01 -1.377672e-01
25%   -3.729927e-02 -4.464164e-02  ... -3.324559e-02 -3.317903e-02
50%    5.383060e-03 -4.464164e-02  ... -1.947171e-03 -1.077698e-03
75%    3.807591e-02  5.068012e-02  ...  3.243232e-02  2.791705e-02
max    1.107267e-01  5.068012e-02  ...  1.335973e-01
1.356118e-01
```

```
[8 rows x 10 columns]
```



RESULT:

Thus the code is executed successfully.

EXP NO: 16 TIME SERIES ANALYSIS OF WEBSITE TRAFFIC

AIM:

To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

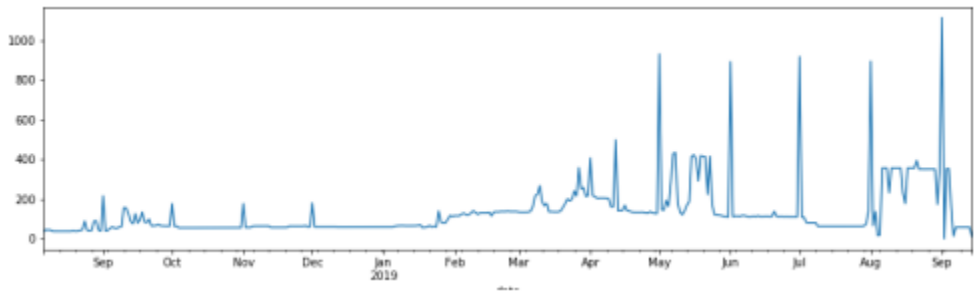
PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

data = pd.read_csv('website_traffic.csv', parse_dates=['Date'], index_col='Date')
decomposition = seasonal_decompose(data['Page_Views'], model='additive')
decomposition.plot()
plt.show()

data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /
data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 17 RANDOM SAMPLING AND SAMPLING DISTRIBUTION

AIM:

To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

PYTHON CODE:

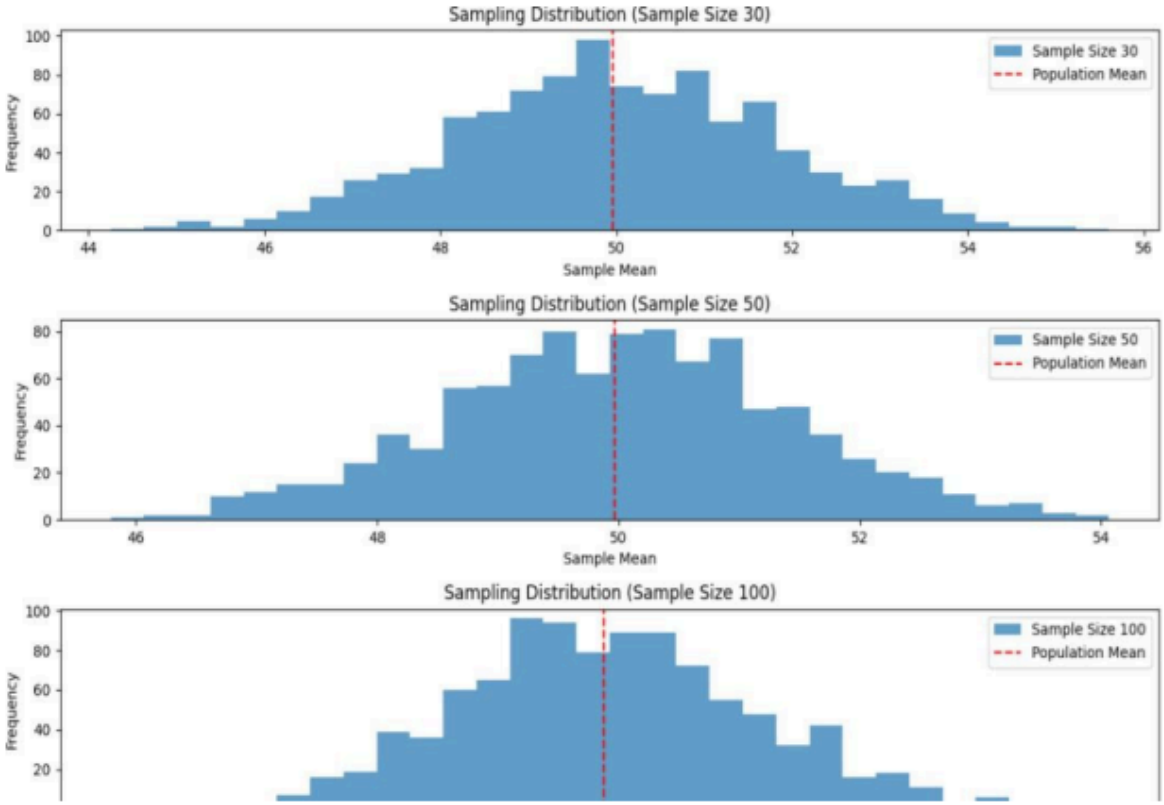
```
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)
sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}
for size in sample_sizes:
sample_means[size] = []
for _ in range(num_samples):
sample = np.random.choice(population, size=size, replace=False)
sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
plt.subplot(len(sample_sizes), 1, i+1)
plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size {size}')
plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5,
label='Population Mean')
plt.title(f'Sampling Distribution (Sample Size {size})')
plt.xlabel('Sample Mean')
plt.ylabel('Frequency')
plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT:

Thus the code is executed successfully.

EXP NO: 18 EXPERIMENT TO UNDERSTAND MATPLOTLIB LIBRARY USE CASES IN DATA SCIENCE THROUGH VISUALIZATION

AIM:

Use Iris data set to understand the Matplot lib library.

PYTHON CODE:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.read_csv('/content/Iris_Dataset.csv')
data.info()
data.describe()
data.value_counts('variety')
sns.countplot(x='variety', data=data)
plt.show()
dummies = pd.get_dummies(data.variety)

FinalDataset = pd.concat(
    [pd.get_dummies(data.variety), data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

FinalDataset.head()
sns.scatterplot(
    x='SepalLengthCm',
    y='SepalWidthCm',
    hue='variety',
    data=data
)
plt.show()
sns.scatterplot(
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='variety',
    data=data
)
plt.show()
```

```
sns.pairplot(data, hue='variety', height=3)
plt.show()

sns.FacetGrid(data, hue='variety', height=5).map(
   sns.histplot, 'PetalLengthCm'
).add_legend()
plt.show()

sns.FacetGrid(data, hue='variety', height=5).map(
   sns.histplot, 'PetalWidthCm'
).add_legend()
plt.show()

sns.FacetGrid(data, hue='variety', height=5).map(
   sns.histplot, 'SepalLengthCm'
).add_legend()
plt.show()

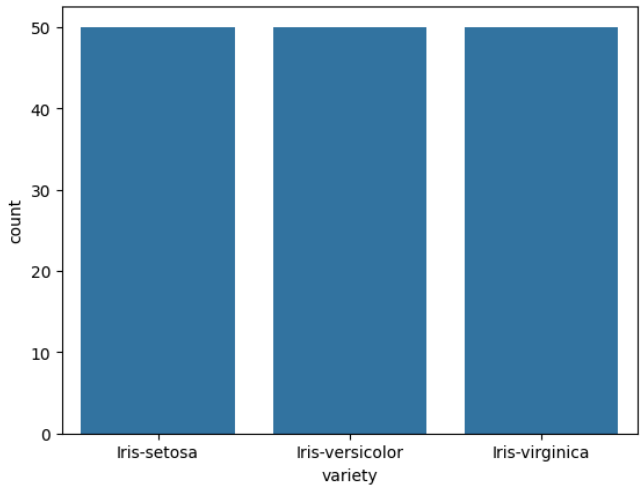
sns.FacetGrid(data, hue='variety', height=5).map(
   sns.histplot, 'SepalWidthCm'
).add_legend()
plt.show()
```

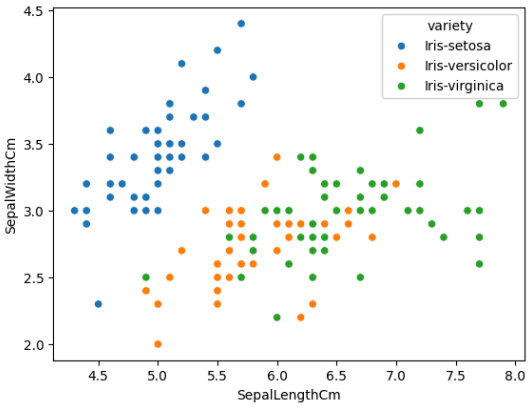
OUTPUT:

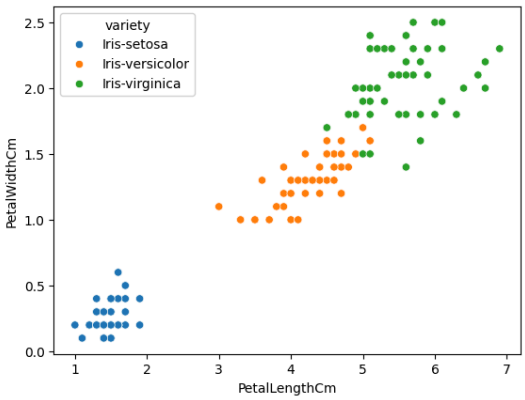
```
     SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm   Id  \
0              5.1           3.5            1.4           0.2    1
1              4.9           3.0            1.4           0.2    2
2              4.7           3.2            1.3           0.2    3
3              4.6           3.1            1.5           0.2    4
4              5.0           3.6            1.4           0.2    5
..             ...           ...            ...           ...  ...
145            6.7           3.0            5.2           2.3  146
146            6.3           2.5            5.0           1.9  147
147            6.5           3.0            5.2           2.0  148
148            6.2           3.4            5.4           2.3  149
149            5.9           3.0            5.1           1.8  150

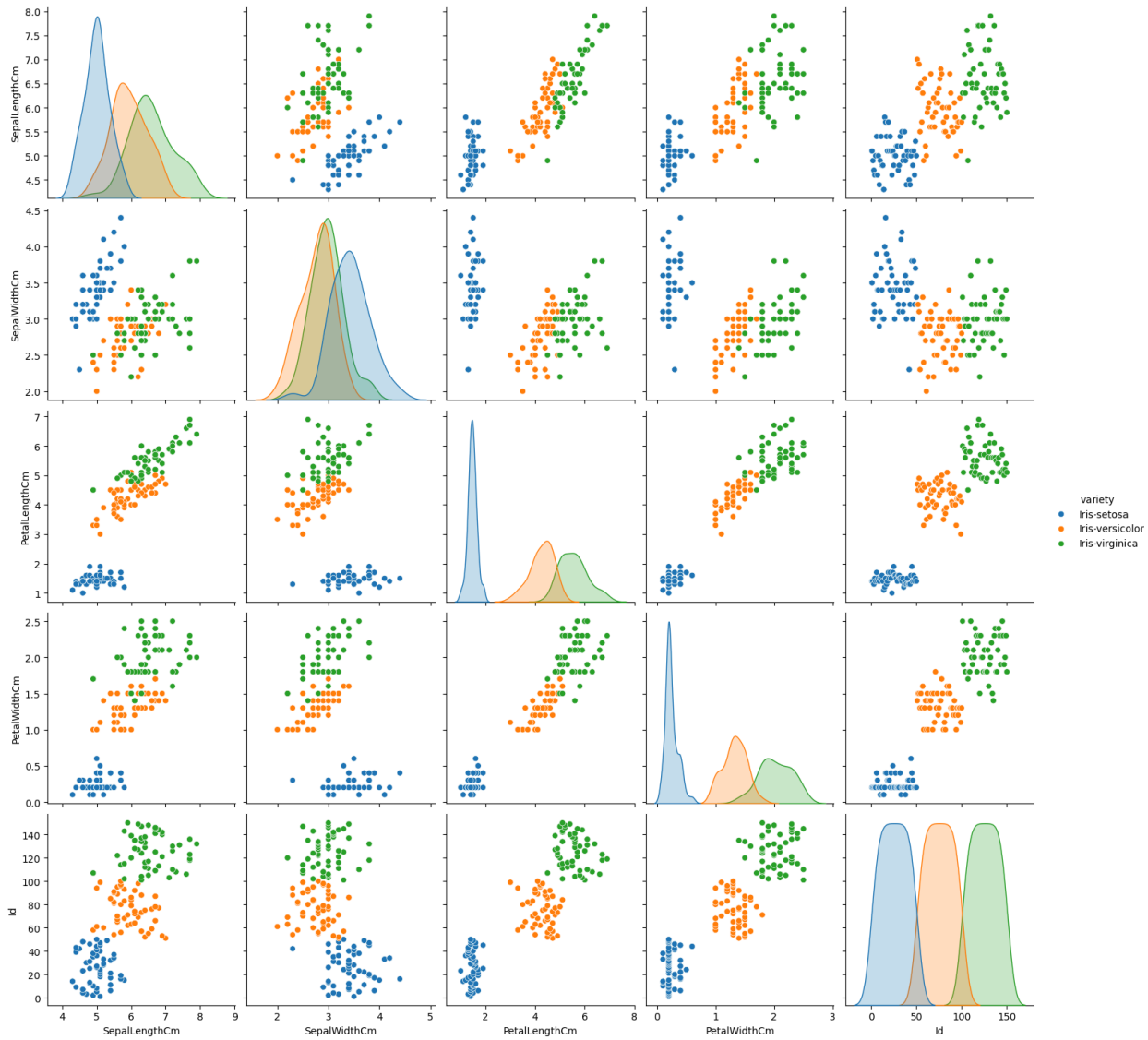
            variety
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

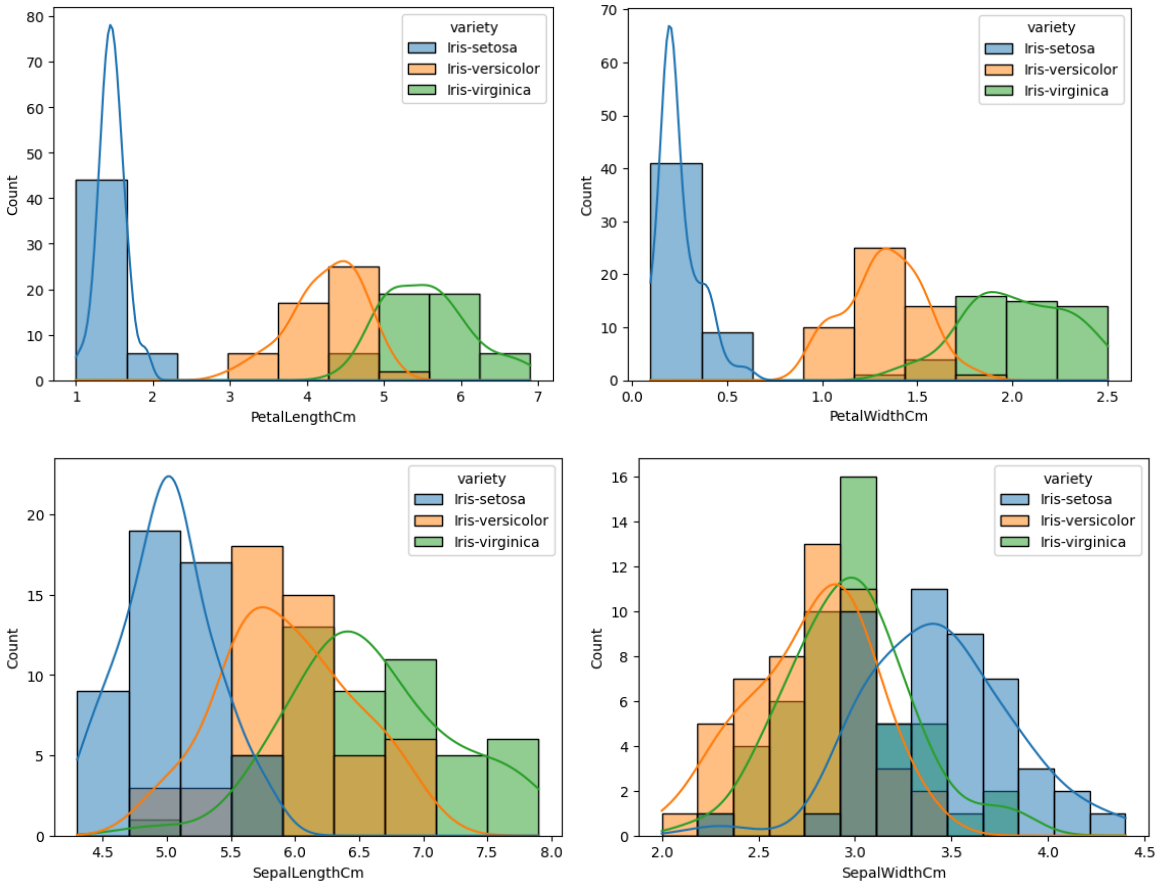
[150 rows x 6 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   SepalLengthCm  150 non-null    float64
 1   SepalWidthCm   150 non-null    float64
 2   PetalLengthCm  150 non-null    float64
 3   PetalWidthCm   150 non-null    float64
 4   Id             150 non-null    int64
 5   variety        150 non-null    category
dtypes: category(1), float64(4), int64(1)
memory usage: 6.3 KB
       SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm          Id
count     150.000000    150.000000     150.000000    150.000000  150.000000
mean        5.843333      3.057333       3.758000      1.199333   75.500000
std         0.828066      0.435866       1.765298      0.762238   43.445368
min         4.300000      2.000000       1.000000      0.100000    1.000000
25%         5.100000      2.800000       1.600000      0.300000   38.250000
50%         5.800000      3.000000       4.350000      1.300000   75.500000
75%         6.400000      3.300000       5.100000      1.800000  112.750000
max         7.900000      4.400000       6.900000      2.500000  150.000000
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```






variety
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
PetalLengthCm


SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
Id
variety
Iris-setosa
Iris-versicolor
Iris-virginica



RESULT:

Thus the code is executed successfully.

EXP NO: 19 EXPERIMENT TO UNDERSTAND ARRAY FUNCTION IN DATA SCIENCE

AIM:

Understand array function using Numpy library.

PYTHON CODE:

```
import numpy as np
array = np.random.randint(1, 100, 9)
array = np.array([83, 25, 19, 47, 62, 15, 96, 39, 51])
print(array)
print(np.sqrt(array))
print(array.ndim)
new_array = array.reshape(3, 3)
print(new_array)
print(new_array.ndim)
print(new_array.ravel())
newm = new_array.reshape(3, 3)
print(newm)
print(newm[2, 1:3])
print(newm[1:2, 1:3])
print(new_array[0:3, 0:0])
print(new_array[0:2, 0:1])
print(new_array[0:3, 0:1])
print(new_array[1:3])
```

OUTPUT:

```
[83 25 19 47 62 15 96 39 51]
[9.11043358 5.         4.35889894 6.8556546  7.87400787 3.87298335
 9.79795897 6.244998   7.14142843]
1
[[83 25 19]
 [47 62 15]
 [96 39 51]]
2
[83 25 19 47 62 15 96 39 51]
[[83 25 19]
 [47 62 15]
 [96 39 51]]
[39 51]
[[62 15]]
[]
[[83]
 [47]]
[[83]
 [47]
 [96]]
[[47 62 15]
 [96 39 51]]
```

RESULT:

Thus the code is executed successfully.

EXP NO: 20 EXPERIMENT TO UNDERSTAND PANDAS LIBRARY USE CASES IN DATA SCIENCE

AIM:

Understand data frame use cases using pandas library.

PYTHON CODE:

```
import numpy as np
import pandas as pd
list = [[1, 'Smith', 50000],
        [2, 'Jones', 60000]]

df = pd.DataFrame(list)
print(df)
df.columns = ['Empd', 'Name', 'Salary']
print(df)
df.info()
df = pd.read_csv("/content/50_Startups.csv")
df.info()
print(df)
print(df.head())
print(df.tail())
df = pd.read_csv("/content/employee.csv")
print(df.head())
print(df.tail())
df.info()
print(df.salary)
print(type(df.salary))
print(df.salary.mean())
print(df.salary.median())
print(df.salary.mode())
print(df.salary.var())
print(df.salary.std())
print(df.describe())
print(df.describe(include='all'))
empCol = df.columns
print(empCol)
emparray = df.values
print(emparray)
employee_DF = pd.DataFrame(emparray, columns=empCol)
print(employee_DF)
```

OUTPUT:

```
   0      1      2
0  1  Smith  50000
1  2  Jones  60000
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        5 non-null      float64
 1   Administration   5 non-null      float64
 2   Marketing Spend  5 non-null      float64
 3   State            5 non-null      object
 4   Profit           5 non-null      float64
dtypes: float64(4), object(1)
memory usage: 332.0+ bytes
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
   R&D Spend  Administration  Marketing Spend       State     Profit
0  165349.20       136897.80        471784.10    New York  192261.83
1  162597.70       151377.59        443898.53  California  191792.06
2  153441.51       101145.55        407934.54     Florida  191050.39
3  144372.41       118671.85        383199.62    New York  182901.99
4  142107.34        91391.77        366168.42     Florida  166187.94
```

```
   emp id                 name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000
   emp id                  name  salary
2       3                SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      object
 2   salary  7 non-null      int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64
<class 'pandas.core.series.Series'>
5714.285714285715
6000.0
0    5000
1    6000
Name: salary, dtype: int64
2571428.5714285714
1603.5674514745463
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000
```

```
          emp id                    name       salary
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000
Index(['emp id', 'name', 'salary'], dtype='object')
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]
  emp id                  name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
2      3               SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

RESULT:

Thus the code is executed successfully.